

Asignatura: PDAW
Entrega 9

ALUMNO: Rodrigo Sánchez Martínez

PROFESOR: Mario Álvarez Fernández

GRUPO: 2º DAW

EVALUACIÓN: 2ª

FECHA: 11/04/2026

CALIFICACIÓN:

Índice

1. Flujo Frontend / Backend	3
Frontend	3
Backend	4
Flujo entre Backend y Frontend	5
2. Medidas de Seguridad Adoptadas	6
Autenticación y Cifrado de Contraseñas	6
Protección contra Ataques Web	7
3. Modularización del Código	8
Organización Modular del Backend	8
Organización Modular del Frontend	9

1. Flujo Frontend / Backend

Lo más importante de este proyecto es establecer una comunicación fluida y estable entre las partes frontend y backend de la plataforma. Este diseño es esencial para garantizar la escalabilidad de la aplicación y que las capas de procesamiento y presentación puedan cambiar de manera independiente.

Frontend

La interfaz frontend ha sido construida usando React como librería principal para permitir la creación de una página única (SPA) que elimina las recargas innecesarias del navegador y mejora la velocidad de respuesta, algo esencial en el entorno de manejo de inventarios en almacenes. El uso de React también permite una arquitectura basada en componentes reutilizables y consistentes en la experiencia del usuario.

Para el diseño visual se ha adoptado la guía de estilo "Soft Industrial".

Elemento de Interfaz	Tecnología / Recurso	Función Técnica
Estructura	HTML5 / JSX	Definición de la semántica de los componentes de React.
Estilo y Diseño	CSS3 / Styled Components	Implementación de la guía de estilo "Soft Industrial" y diseño responsive.
Iconografía	Lucide-React	Provisión de símbolos visuales claros y vectoriales para acciones rápidas.
Escáner de QR	Librería compatible con JS	Acceso a la API nativa de la cámara del dispositivo móvil para decodificación.
Gestión de Estado	React Hooks (useState, useEffect)	Manejo de datos locales y sincronización con la API en tiempo real.

La navegación de la plataforma se ha diseñado para minimizar la carga mental en los usuarios y administradores. Para cada proceso fundamental se han identificado unas vistas clave que tienen un usuario objetivo concreto.

Vista	Público Objetivo	Funcionalidad Principal
Dashboard Principal	Administrador / Auditor	Visualización de KPIs, niveles de stock global y alertas críticas.
Módulo de Escaneo	Operador	Interfaz optimizada para el uso de la cámara y registro de movimientos rápidos.
Gestión de Catálogo	Administrador	Operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre los registros de productos.
Reportes ABC	Administrador / Auditor	Análisis de rotación de inventario basado en el principio de Pareto.
Perfil de Usuario	Todos los Actores	Gestión de credenciales, preferencias y seguridad de la cuenta.

Backend

El servidor de la aplicación utiliza Python con frameworks como Flask o Django para gestionar la lógica interna y la exposición de servicios a través de una API REST. Esta capa se responsabiliza de procesar las peticiones del frontend, validar la integridad de los datos y asegurar que cada movimiento se registre adecuadamente en la base de datos.

El backend asume la carga de procesamiento pesado incluyendo la generación de códigos QR de forma automática. Haciendo uso de la librería qrcode de Python el sistema es capaz de transformar IDs únicos en imágenes vectoriales SVG o mapas de bits PNG.

La persistencia de datos se gestiona mediante MySQL, elegido por su capacidad de manejar transacciones complejas con integridad referencial. El diseño de la base de datos se estructura en 5 tablas principales:

- USUARIO
- PRODUCTO
- MOVIMIENTO
- ALERTA
- ROL

Flujo entre Backend y Frontend

La interacción entre ambas capas se produce mediante los siguientes pasos:

1. **Captura en Frontend:** el operario escanea el código QR utilizando el componente de la cámara de la aplicación React y la librería de escaneo decodifica el ID del producto.
2. **Petición API:** el frontend envía una petición POST al endpoint de `/api/movimientos` incluyendo el ID extraído, la cantidad y el tipo de movimiento, autenticada mediante un token de inicio de sesión.
3. **Procesamiento en Backend:** el controlador en Python recibe la petición, valida los permisos del rol del usuario y comprueba la existencia del producto en la base de datos.
4. **Transacción de datos:** si la validación es positiva, se actualiza el campo `stock_actual` en la tabla `PRODUCTO` y se inserta un nuevo registro en la tabla `MOVIMIENTO`.
5. **Respuesta y feedback:** el backend devuelve un código de estado 201 exitoso para comunicar la creación correcta. El frontend actualiza la interfaz en tiempo real proporcionando feedback visual.

Este ciclo asegura que no surjan discrepancias entre el almacén físico y el registro digital, permitiendo así una visión en tiempo real que optimiza la rentabilidad para las PyMEs.

2. Medidas de Seguridad Adoptadas

La seguridad en un sistema de gestión de inventarios destinado a pequeñas y medianas empresas (en adelante PYMEs) es fundamental no solo para la protección de los activos físicos, sino también para garantizar el cumplimiento de la normativa vigente (incluyendo el RGPD) y la privacidad de la información estratégica de la empresa. Para minimizar riesgos internos y externos se han implementado diferentes capas de defensa.

Se considera que cada punto de entrada de datos en el sistema podría ser potencialmente inseguro. Se aplican exigentes validaciones en el frontend y en el backend. En el backend, existe una capa de validación de entradas que limpia y comprueba la información antes de que ésta entre en contacto con la lógica de negocio o la base de datos.

- Tipado estricto: Valores numéricos no negativos (en especial en precios y cantidades de stock).
- Limpieza de entradas: Se eliminan caracteres que puedan dar pie a Inyección SQL o Cross-Site Scripting (XSS). De este modo, se asegura que los datos almacenados en MySQL son íntegros.
- Validación de negocio: El sistema impide que se produzcan acciones que violen la lógica de negocio: no es posible intentar retirar más unidades de las que existen (se lanzan avisos visuales inmediatos al usuario).

Autenticación y Cifrado de Contraseñas

La seguridad de las cuentas de usuario está elevada a partir del almacenamiento ilegible, que guarda las credenciales. Según los estándares de la industria, las contraseñas no se almacenan nunca como texto plano.

El sistema utiliza algoritmos de hashing modernos (por ejemplo, Bcrypt) para transformar las contraseñas. En el registro o el cambio de contraseña, el texto plano se transforma en un hash irreversible por medio de una técnica de "salting" (adición de datos aleatorios a la entrada a procesar antes de hacerle el hash), lo que protege contra ataques mediante tablas arcoíris y ataques de fuerza bruta. Para impedir el uso de un ataque mediante tablas arcoíris y el ataque por fuerza bruta, la infraestructura en la nube (VPS) también dispone de certificados SSL para implementar la codificación del tráfico de datos entre los dispositivos móviles y el servidor, así como la modificación del texto de las contraseñas. La codificación asegura que la información no sea interceptada cuando la información es enviada.

Protección contra Ataques Web

El sistema tiene específicos mecanismos para mitigar vulnerabilidades extensamente conocidas en aplicaciones web:

- Protección CSRF: se utiliza el mecanismo de tokens de seguridad válidos en las consultas de modificación de datos. Asegura que la operación (ejemplo: eliminar un producto) pues será ejecutada desde la interfaz de la aplicación web y no desde un sitio web de carácter malicioso.
- Rate limiting: el servidor restringe el número de consultas permitidas por minuto desde la misma dirección IP. Se previene el abuso por medio de la API, se contrarrestan ataques DoS y se protegen formularios de login contra ataques de tipo brute force.
- Gestión segura de la sesión: los tokens de autenticación se gestionan con atributos de seguridad (HttpOnly y Secure) para prevenir su robo a través de scripts maliciosos en el navegador.

3.Modularización del Código

Se ha elaborado la arquitectura del software aplicando los principios establecidos para la modularidad, de forma que se obtenga mantenibilidad, se haga más fácil el trabajo en equipo y que el propio sistema pueda crecer de forma no demasiado arbitraria y no generando una cantidad excesiva de deuda técnica. La separación de responsabilidades evita que no haya lógica escrita en un único recurso, ayudando a encontrar errores y a corregirlos más rápidamente.

Organización Modular del Backend

En el lado servidor se usa el patrón de Blueprints para dividir la aplicación en módulos funcionales independientes el uno del otro.

Módulo / Fichero	Responsabilidad técnica	Beneficio de usar como módulo
auth.py	Gestión de login, registro, sesiones y hashing de contraseñas.	Aísla la lógica de seguridad, facilitando su auditoría independiente.
api.py	Definición de los endpoints principales de la API REST.	Centraliza la comunicación con el frontend.
database.py	Configuración de la conexión a MySQL y modelos del ORM.	Permite cambiar de motor de base de datos con un impacto mínimo.
products.py	Lógica del CRUD de catálogo y generación de códigos QR.	Facilita la actualización de la lógica de negocio sin afectar a la autenticación.
decorators.py	Funciones para control de acceso por roles y logging.	Permite reutilizar lógica de seguridad en cualquier ruta de forma limpia.

Esta estructura modular permite que el sistema sea escalable por si en el futuro se desean añadir módulos adicionales o modificar existentes.

Organización Modular del Frontend

La modularización de la aplicación frontend se manifiesta de la siguiente forma:

- Componentes Reutilizables: Elementos como el lector de QR, las tarjetas de producto o las tablas de movimientos se crean a partir de componentes independientes. Estos componentes se guardan en subcarpetas de la carpeta /components, evitando así duplicaciones y garantizando una determinada forma de acceso a la interfaz.
- Hooks personalizados: La lógica de interacción con importantes paradigmas de frontend (ej. la interacción con la cámara, la realización de peticiones fetch) se extrae a "hooks", a los cuales puede acceder cualquier parte de la aplicación haciendo uso de un acceso limpio e instintivo.
- Peticiones a servicios: Las peticiones a los servicios de backend que se realizan desde la aplicación son centralizadas en una capa de servicios, lo que permite realizar modificaciones en la URL del servidor o en una estructura de JSON recibida desde el servidor desde un único lugar en la aplicación.
- Enrutamiento del lado del cliente: Existe la estructura de rutas (ej. React Router) que permite combinar las urls en la aplicación con sus correspondientes componentes de página, proporcionando así una experiencia de navegación fluida y rápida.